

北京林业大学

数据库原理与应用

数据库完整性含义、 描述方式及分类方式

本章目录 CONTENTS

- | 数据库完整性含义
- | 数据库完整性的描述方式
- | 数据库完整性约束的分类
- | 总结

数据库完整性的含义



数据库完整性的定义

数据库的完整性



确保数据库
中的数据

数据的正确性

数据的有效性

数据的相容性



数据库完整性的定义

在安全性的基础上，完整性可以理解为：

- 防止合法用户使用数据库时向数据库中加入不符合语义的数据。
- 完整性措施的防范对象是不合语义的数据。

数据库完整性的 描述方式



使用规则描述数据库完整性的方法

描述完整性的3个部分

触发条件：规定系统什么时候使用规则来检查数据。

WHEN

约束条件：规定系统检查用户发出的操作请求违背了什么样的完整性约束条件。

Condition

违约响应：规定系统如果发现用户发出的操作请求违背了完整性约束条件，应该采取一定的动作来保证数据的完整性，即违约时要做的事情。

What



规则的执行时间

规则的执行时间



立即执行：在执行多个语句构成的事务时，执行完用户事务的一条语句后，系统立即对数据进行完整性条件检查。



延迟执行：执行完事务的所有语句后，系统才数据进行完整性条件检查。



完整性的形式化描述

一条完整性可以用一个五元组 (D, O, A, C, P) 来形式化地表示

D (Data) :

代表约束作用的数据对象。可以是关系、元组和列三种对象；

O (Operation)

代表触发完整性检查的数据库操作，即当用户发出什么操作请求时需要检查该完整性规则，是立即执行还是延迟执行；

A (Assertion)

代表数据对象必须满足的语义约束，这是规则的主体；



完整性的形式化描述

C (Condition)

代表选择A作用的数据对象值的谓词;

P (Procedure)

代表违反完整性规则时触发执行的操作过程。



完整性的形式化举例



例如，对于“学号(SNo)不能为空”的这条完整性约束中，D、O、A、C、P的含义分别如下：

- D：代表约束作用的数据对象为SNo属性；
- O：当用户插入或修改数据时需要检查该完整性规则；
- A：SNo不能为空；
- C：A可作用于所有记录的SNo属性；
- P：拒绝执行用户请求。

数据库完整性 约束的分类



数据库完整性约束的分类

根据约束条件：值的约束和结构的约束



值的约束即对数据类型、数据格式、取值范围和空值等进行规定

(1) 对数据类型的约束，包括数据的类型、长度、单位和精度等。

例如：规定学生的姓名数据类型为字符型，长度为8



数据库完整性约束的分类



(2) 对数据格式的约束。
例如：规定出生日期的格式为YY.MM.DD



(3) 对取值范围的约束。
例如：月份的取值范围为1 ~ 12



(4) 对空值的约束，空值表示未定义或者未知数值。例如，在SC关系中，学号和课程号不可以为空。



数据库完整性约束的分类

结构的约束即对数据之间联系的约束

(1) 函数依赖约束。

例如：2NF，3NF和BCNF需要满足的函数依赖。

(2) 实体完整性约束。

例如：在学生关系中，主码学号不能为空。

(3) 参照完整性约束。

例如：在学生关系中，外码系别的约束。

(4) 统计约束。

例如：规定某一个的数值不得高于这一列的平均值等。



数据库完整性约束的分类

根据约束状态：静态约束和动态约束

静态约束

- 静态约束是指对数据库每一个确定状态所应满足的约束条件，是反映数据库状态合理性的约束，这是最重要的一类完整性约束。
- 值的约束和结构的约束均属于静态约束。



数据库完整性约束的分类

根据约束状态：静态约束和动态约束

动态约束

- 动态约束是指数据库从一种状态转变为另一种状态时，新旧值之间所应满足的约束条件，动态约束反映的是数据库状态变迁的约束。
- 例如，学生年龄在更改时只能增长，职工工资在调整时不得低于其原来的工资。

总结



知识点总结

数据库完整性

含义

描述方式

形式化表达

分类

触发条件

约束条件

违约条件

D, O, A, C, P

值和结构

静态和动态

数据库完整性的实施

本章目录 CONTENTS

➤ | 规则

➤ | 默认

➤ | 总结

规则



使用规则进行数据库完整性控制



规则（Rule）就是数据库对存储在表中的列或用户自定义数据类型中的值的规定和限制。



规则与其作用的表或用户自定义数据类型是相互独立的，即表或用户自定义对象的删除、修改不会对与之相连的规则产生影响。



规则和约束可以同时使用，表的列可以有一个规则及多个check约束。



规则的使用步骤

1. 创建规则
2. 将规则绑定到数据库的对象上，或者将规则从数据库的对象上松绑。

创建规则

```
CREATE RULE rule_name AS  
condition_expression
```



规则的使用步骤

【例】创建学生年龄规则

```
CREATE RULE age_rule
```

```
AS @age >= 18 and @age <= 50
```



规则的使用步骤

1. 创建规则
2. 将规则绑定到数据库的对象上，或者将规则从数据库的对象上松绑。

□创建规则后，规则仅仅是一个存在于数据库中的对象，并未发生作用。需要将规则与数据库表或用户自定义对象联系起来，才能达到创建规则的目的。

□所谓绑定就是指定规则作用于哪个表的哪一列或哪个用户自定义数据类型。

□解除规则与对象的绑定称为“松绑”。



绑定规则和松绑规则的方法

用存储过程sp_bindrule绑定规则

- `sp_bindrule[@rulename=]`
`'rule',[@objname =] 'object_name'[,`
`'futureonly']`
- [例] 绑定规则age_rule 到S表的字段Age。
`EXEC sp_bindrule 'age_rule', 'S.Age'`

规则对已经输入表中的数据不起作用。



绑定规则和松绑规则的方法



用存储过程sp_unbindrule松绑规则的绑定
sp_unbindrule [@objname =]
'object_name'[, 'futureonly']
[例] 解除已绑定到S表的字段Age的规则
age_rule。
EXEC sp_unbindrule 'S.Age'



删除规则的方法

删除规则

□使用DROP RULE命令删除规则

```
DROP RULE {rule_name} [...n]
```

[例] 删除age_rule规则。

```
DROP RULE age_rule
```



在删除一个规则前必须先将其绑定的对象解除绑定。



默认



默认的定义及创建方法

- 默认 (Default) 是向用户输入记录时没有指定具体数据的列中自动插入的数据。
- 表的一列或一个用户自定义数据类型只能与一个默认绑定。



默认的定义及创建方法

默认的使用步骤

1. 创建默认
2. 绑定默认到数据库中的对象



默认的定义及创建方法

创建默认

```
CREATE DEFAULT default_name  
AS constant_expression
```

[例] 创建出生日期默认birthday_defa。

```
CREATE DEFAULT birthday_defa  
AS '1978-1-1'
```



默认的查看、绑定和松绑

查看默认

使用sp_helptext存储过程可以查看默认的信息。

[例] 查看默认birthday_defa。

```
EXEC sp_helptext birthday_defa
```



默认的查看、绑定和松绑

默认的绑定与松绑

□用存储过程sp_bindefault 绑定默认

```
sp_bindefault [@defname =] 'default',  
              [@objname =] 'object_name'
```

```
[, 'futureonly']
```

□用存储过程sp_unbindefault 解除默认的绑定

```
sp_unbindefault [@objname =] 'object_name'  
[, 'futureonly']
```



默认的绑定和松绑的举例、删除默认

[例]绑定默认birthday_defa到数据表S的Birthday列上。

```
EXEC sp_bindefault birthday_defa, 'S.[Birthday]'
```

[例]解除默认birthday_defa与表S的Birthday列的绑定

```
EXEC sp_unbindefault 'S.[Birthday]'
```



默认的绑定和松绑的举例、删除默认

删除默认

- 使用DROP DEFAULT命令删除默认
DROP DEFAULT{default_name}[,...n]
- [例5-21]删除学生生日默认birthday_defa.
DROP DEFGAULT birthday_defa

在删除一个默认前必须先将其绑定的对象解除绑定

总结



知识点总结

数据库完整性的实施

使用规则

使用默认